

1. FCFS Scheduling algorithm

```
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], wt[20], tat[20];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time for each process:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
            i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

Output:

```
vishnu@VISHNU:~$ nano fcfs.c
vishnu@VISHNU:~$ gcc fcfs.c -o fcfs
vishnu@VISHNU:~$ ./fcfs
Enter number of processes: 4
Enter burst time for each process:
P1: 6
P2: 8
P3: 6
P4: 5
```

Process	BT	WT	TAT
P1	6	0	6
P2	8	6	14
P3	6	14	20
P4	5	20	25

2. SJF_NONpreemptive

```

#include <stdio.h>

int main() {
    int n, i, j;
    int bt[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Sort burst times
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (bt[i] > bt[j]) {
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
            i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}

```

Output:

```
vishnu@VISHNU:~$ nano sjf_nonpreemptive.c
vishnu@VISHNU:~$ gcc sjf_nonpreemptive.c -o sjf_nonpreemptive
vishnu@VISHNU:~$ ./sjf_nonpreemptive
Enter number of processes: 5
Enter burst time:
P1: 6
P2: 7
P3: 9
P4: 7
P5: 5

Process BT      WT      TAT
P1      5        0        5
P2      6        5       11
P3      7       11       18
P4      7       18       25
P5      9       25       34

Average Waiting Time = 11.80
Average Turnaround Time = 18.60
```

3. Priority

```
#include <stdio.h>

int main() {
    int n, i, j;
    int bt[20], priority[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time and priority:\n");

    for (i = 0; i < n; i++) {
        printf("P%d Burst Time: ", i + 1);
        scanf("%d", &bt[i]);

        printf("P%d Priority: ", i + 1);
        scanf("%d", &priority[i]);
    }

    // Sort according to priority
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {

            if (priority[i] > priority[j]) {

                temp = priority[i];
                priority[i] = priority[j];
```

```

        priority[j] = temp;

        temp = bt[i];
        bt[i] = bt[j];
        bt[j] = temp;
    }
}

// Waiting time
wt[0] = 0;

for (i = 1; i < n; i++) {
    wt[i] = wt[i - 1] + bt[i - 1];
}

// Turnaround time
for (i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

printf("\nProcess\tBT\tPriority\tWT\tTAT\n");

for (i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t\t%d\t%d\n",
        i + 1, bt[i], priority[i], wt[i], tat[i]);
}

return 0;
}

```

```

vishnu@VISHNU:~$ nano priority.c
vishnu@VISHNU:~$ gcc priority.c -o priority
vishnu@VISHNU:~$ ./priority
Enter number of processes: 5
Enter burst time and priority:
P1 Burst Time: 4
P1 Priority: 6
P2 Burst Time: 5
P2 Priority: 4
P3 Burst Time: 1
P3 Priority: 1
P4 Burst Time: 2
P4 Priority: 3
P5 Burst Time: 5
P5 Priority: 8

Process BT      Priority      WT      TAT
P1      1        1          0        1
P2      2        3          1        3
P3      5        4          3        8
P4      4        6          8       12
P5      5        8         12       17
vishnu@VISHNU:~$ |

```

Output:

4. ROUND ROBIN

```

#include <stdio.h>

int main() {
    int n, i;
    int bt[20], rem_bt[20];
    int wt[20], tat[20];
    int quantum;
    int time = 0;
    int done;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);

        rem_bt[i] = bt[i];
        wt[i] = 0;
    }
}

```

```

printf("Enter time quantum: ");
scanf("%d", &quantum);

// Round Robin
while (1) {

    done = 1;

    for (i = 0; i < n; i++) {

        if (rem_bt[i] > 0) {

            done = 0;

            if (rem_bt[i] > quantum) {
                time = time + quantum;
                rem_bt[i] = rem_bt[i] - quantum;
            }
            else {
                time = time + rem_bt[i];

                wt[i] = time - bt[i];

                rem_bt[i] = 0;
            }
        }
    }

    if (done == 1)
        break;
}

// Turnaround time
for (i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

printf("\nProcess\tBT\tWT\tTAT\n");

for (i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\n",
        i + 1, bt[i], wt[i], tat[i]);
}

return 0;
}

```

Output:

```
vishnu@VISHNU:~$ nano roundrobin.c
vishnu@VISHNU:~$ gcc roundrobin.c -o roundrobin
vishnu@VISHNU:~$ ./roundrobin
Enter number of processes: 7
Enter burst time:
P1: 8
P2: 8
P3: 55
P4:
2
P5: 2
P6: 2
P7: 2
Enter time quantum: 6

Process BT      WT      TAT
P1      8      20      28
P2      8      22      30
P3     55     24     79
P4      2     18     20
P5      2     20     22
P6      2     22     24
P7      2     24     26
vishnu@VISHNU:~$ |
```